

State Management Modern & Arsitektur Clean Code

Pengelolaan Alur Data, Pemisahan Logika Bisnis, dan Penanganan Siklus UI pada Aplikasi NutriMarket

Mata Kuliah / CPMK	CPMK 3: State Management Modern & Arsitektur Clean Code
Proyek Implementasi	NutriMarket (clev_ai) - Marketplace Makanan Sehat & AI Nutritionist
Pustaka State Management	Provider v6.1.2 (Standar Industri Resmi Flutter / Google)
Pola Arsitektur	Clean Architecture 3-Tier Layering (Presentation, Domain, Data)
Target Platform & Tech Stack	Flutter SDK (Dart 3.2+), Supabase Cloud Backend, Encrypted Local Cache

Ringkasan Eksekutif:

Dokumen ini menyajikan bukti implementasi dan penjelasan teknis mendalam mengenai arsitektur perangkat lunak pada aplikasi **NutriMarket**. Pembahasan mencakup 3 aspek utama panduan pengerjaan: (1) Pemilihan pustaka State Management standar industri **Provider**, (2) Penerapan **Clean Architecture** dengan pemisahan tegas antara Layer Presentation, Domain/Business Logic, dan Data, serta (3) Pengendalian interaksi UI terhadap status dinamis (Initial, Loading, Empty, Success, Failure) pada alur Keranjang Belanja (*Cart Flow*), Langganan Premium (*Subscription/Paywall Flow*), dan Katalog Produk. Seluruh bagian dilengkapi dengan rujukan lokasi direktori dan cuplikan kode sumber nyata dari proyek.

1. PILIHAN PUSTAKA STATE MANAGEMENT STANDAR INDUSTRI: PROVIDER

1.1 Alasan Pemilihan & Karakteristik Provider

Untuk mengelola *state* aplikasi secara efisien dan terstruktur, proyek **NutriMarket** menggunakan pustaka **Provider (v6.1.2)**. Provider merupakan pustaka manajemen state resmi yang direkomendasikan secara langsung oleh tim pengembang Flutter di Google karena alasan-alasan teknis berikut:

- Standar Industri & Kompatibilitas Ekosistem:** Provider membungkus mekanisme bawaan `InheritedWidget` menjadi API yang sangat intuitif, deklaratif, dan aman dari *memory leak*.
- Implementasi Observer Pattern:** Memisahkan antara subjek penyedia data (`ChangeNotifier`) dengan pengamat (*Widget listener*), sehingga mutasi data tidak memerlukan manipulasi langsung pada widget UI.
- Pembersihan & Optimasi Rebuild:** Provider hanya me-rebuild sub-tree widget yang secara eksplisit mendengarkan perubahan data, mencegah *jank* dan rebuild global yang boros sumber daya.
- Testability:** Service layer yang dibuat terlepas dari konteks UI, mempermudah pengujian *Unit Testing* tanpa perlu mem-boot simulator UI.

1.2 Registrasi Sentral MultiProvider pada Root Aplikasi

Seluruh penyedia layanan (*service providers*) didaftarkan pada root widget tree menggunakan `MultiProvider` di file utama aplikasi:

Lokasi Kode: `lib/main.dart` (Baris 67 - 88)

```

class HealthCarePlusApp extends StatelessWidget {
  const HealthCarePlusApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiProvider(
      providers: [
        // Remote & Data Storage Layer Providers
        ChangeNotifierProvider<SupabaseService>.value(value: SupabaseService()),
        ChangeNotifierProvider<SecureStorageService>.value(value: SecureStorageService()),
        ChangeNotifierProvider<OfflineSyncService>.value(value: OfflineSyncService()),

        // Domain & Business Logic Service Providers
        ChangeNotifierProvider<ProductRepository>.value(value: ProductRepository()),
        ChangeNotifierProvider<HistoryService>.value(value: HistoryService()),
        ChangeNotifierProvider<CartService>.value(value: CartService()),
        ChangeNotifierProvider<OrderService>.value(value: OrderService()),
        ChangeNotifierProvider<FavoriteService>.value(value: FavoriteService()),
        ChangeNotifierProvider<NotificationService>.value(value: NotificationService()),
      ],
      child: MaterialApp(
        title: 'Nutri Market',
        theme: AppTheme.lightTheme,
        home: const SplashScreen(),
      ),
    );
  }
}

```

1.3 Pola Resolusi State di Widget (Watch vs Read vs Consumer)

Metode Resolusi	Mekanisme Kerja	Contoh Pemakaian di Proyek
<code>context.watch<T>()</code>	Mendaftarkan widget sebagai <i>active subscriber</i> . Widget akan di-rebuild otomatis saat <code>notifyListeners()</code> terpanggil.	<code>HomeScreen</code> mendengarkan update daftar katalog produk di <code>ProductRepository</code> .
<code>context.read<T>()</code>	Mengambil referensi instance service tanpa mendaftarkan listener (tidak memicu rebuild).	Dipanggil dalam event handler tombol aksi: <code>onPressed: () => context.read<CartService>().addToCart(p)</code> .
<code>AnimatedBuilder</code> / <code>Consumer</code>	Membungkus area spesifik widget tree yang membutuhkan update lokal.	Badge counter keranjang belanja dan kartu total kalkulasi harga di <code>CartScreen</code> .

2. PENERAPAN STRUKTUR CLEAN ARCHITECTURE (3 LAYERS)

Arsitektur aplikasi NutriMarket dirancang dengan prinsip **Clean Architecture** untuk memisahkan tanggung jawab antarmuka, aturan bisnis, dan akses data. Pembagian lapisannya adalah sebagai berikut:

1. Presentation Layer (UI & Widgets)

Lokasi: `lib/features/*/screens/` & `lib/features/*/widgets/`

Bertanggung jawab mutlak atas visualisasi data, tata letak antarmuka, animasi, serta penerimaan input pengguna. Layer ini **tidak boleh** mengandung query database atau logika bisnis langsung; seluruh aksi pengguna didelegasikan ke Domain Layer melalui Provider.

2. Domain / Business Logic Layer (Services & Repositories)

Lokasi: `lib/features/*/services/`

Menampung seluruh aturan bisnis aplikasi: kalkulasi harga subtotal dan ongkos kirim, validasi status kupon/diskon premium, mutasi keranjang belanja, manipulasi status pesanan, serta manajemen antrian sinkronisasi offline. Menggunakan `ChangeNotifier` untuk memicu pembaruan state reaktif.

3. Data Layer (Models, Local Storage, & Remote API)

Lokasi: `lib/data/` & `lib/features/*/models/`

Mengelola akses data mentah baik dari cloud (Supabase PostgreSQL & Auth) maupun perangkat lokal (SharedPreferences & FlutterSecureStorage). Mengubah data JSON/Map menjadi entity model Dart dan menangani caching persisten offline-first.

2.1 Matriks Komponen & Pemetaan File Clean Architecture

Layer	Komponen / Kelas	Path Direktori	Tanggung Jawab Utama
Presentation	CartScreen	<code>lib/features/cart/screens/cart_screen.dart</code>	Render list item keranjang, nota rincian, dan tombol checkout.
	PaywallScreen	<code>lib/features/subscription/screens/paywall_screen.dart</code>	Render paket langganan NutriMarket Plus & benefit card.
	HomeScreen & FoodCard	<code>lib/features/products/screens/home_screen.dart</code>	Tampilan katalog makanan sehat, banner gizi, dan search filter.
Domain / Business	CartService	<code>lib/features/cart/services/cart_service.dart</code>	Logika manipulasi item belanja, diskon, hitung total harga.
	OrderService	<code>lib/features/orders/services/order_service.dart</code>	Aturan pembuatan order, status checkout (PAID/UNPAID).
	HistoryService	<code>lib/features/orders/services/history_service.dart</code>	Kelola profil pengguna, status langganan <code>isPremium</code> , AI chat.
Data Layer	SupabaseService	<code>lib/data/remote/supabase_service.dart</code>	Komunikasi REST API PostgreSQL & Supabase Auth.
	LocalStorageService	<code>lib/data/local/local_storage_service.dart</code>	Penyimpanan cache offline via <code>SharedPreferences</code> .
	FoodProduct & CartItem	<code>lib/features/cart/models/cart_item.dart</code>	Data Transfer Object (DTO) & parsing JSON mapping.

2.2 Bukti Pemisahan Logika Bisnis dari UI pada CartService

Potongan kode berikut membuktikan bahwa perhitungan harga, penambahan item, persistensi cache lokal, dan antrian sinkronisasi cloud diproses murni di Domain Layer tanpa menyentuh widget UI:

Lokasi Kode: `lib/features/cart/services/cart_service.dart` (Baris 14 - 32, 113 - 153)

```

class CartService extends ChangeNotifier {
  final List<CartItem> _items = [];

  List<CartItem> get items => List.unmodifiable(_items);
  int get itemCount => _items.fold(0, (sum, item) => sum + item.quantity);
  double get subtotal => _items.fold(0, (sum, item) => sum + item.totalPrice);
  double get deliveryFee => _items.isEmpty ? 0 : 10000;
  double get total => subtotal + deliveryFee;

  // Logika Bisnis: Tambah Produk ke Keranjang
  void addToCart(FoodProduct product, {int quantity = 1, String sellerNotes = ''}) {
    final index = _items.indexWhere((item) => item.product.id == product.id);
    if (index >= 0) {
      _items[index].quantity += quantity;
    } else {
      _items.add(CartItem(product: product, quantity: quantity, sellerNotes: sellerNotes));
    }

    // 1. Simpan ke Cache Lokal (Data Layer)
    _persistLocalCart();

    // 2. Masukkan ke Antrean Sinkronisasi Cloud Offline-First
    LocalStorageService().addToSyncQueue({
      'id': 'task_cart_${product.id}',
      'type': 'cart_item',
      'payload': {'product_id': product.id, 'quantity': quantity},
    });
    OfflineSyncService().triggerSync();

    // 3. Pemicu Update Reaktif UI
    notifyListeners();
  }
}

```

3. INTERAKSI UI & PENANGANAN SIKLUS STATE (INITIAL, LOADING, SUCCESS, ERROR)

Setiap modul interaksi pengguna pada aplikasi NutriMarket mengimplementasikan penanganan status secara komprehensif. Berikut adalah dokumentasi terperinci untuk 3 alur utama:

3.1 Studi Kasus 1: Alur Keranjang Belanja & Checkout (Cart Flow)

State UI	Kondisi Logika Sistem	Representasi Visual Antarmuka	Lokasi Implementasi Kode
INITIAL	Aplikasi membaca data keranjang tersimpan dari <code>SharedPreferences</code> saat pertama kali start.	Keranjang otomatis terisi item sesi sebelumnya tanpa reset.	<code>CartService.initFromLocalStorage()</code> <code>lib/features/cart/services/cart_service.dart:46</code>
EMPTY	Kondisi ketika <code>_cartService.items.isEmpty</code> bernilai <code>true</code> .	Tampilan ilustrasi tas belanja kosong, pesan informatif, dan tombol 'Mulai Belanja Sehat'.	<code>_buildEmptyCart(isDark)</code> <code>lib/features/cart/screens/cart_screen.dart:98, 126</code>
SUCCESS (LOAD)	Data item tersedia dalam list <code>_items</code> . Subtotal dan biaya dihitung reaktif.	List kartu item belanja (<code>_buildCartItemCard</code>), rincian harga, dan ringkasan diskon.	<code>AnimatedBuilder(animation: _cartService)</code> <code>lib/features/cart/screens/cart_screen.dart:94-118</code>
LOADING	Proses sinkronisasi data keranjang ke server Supabase Cloud saat online.	Background sync berjalan via <code>OfflineSyncService</code> tanpa memblokir UI pengguna.	<code>CartService.syncWithSupabase()</code> <code>lib/features/cart/services/cart_service.dart:95</code>
SUCCESS (CHECKOUT)	Pembayaran QRIS / VA berhasil diverifikasi, pesanan dibuat ke <code>OrderService</code> , keranjang di-clear.	Notifikasi sukses <code>AppToast.success</code> , keranjang ter-reset, dan dialihkan ke Riwayat Pesanan.	<code>_executeOrderSuccess()</code> <code>lib/features/orders/screens/checkout_screen.dart:323</code>
ERROR / FAILURE	Validasi checkout gagal (alamat belum diisi atau keranjang belanja kosong).	Toast merah peringatan dengan pesan kesalahan yang jelas.	<code>AppToast.show(type: ToastType.error)</code> <code>lib/features/orders/screens/checkout_screen.dart</code>

Cuplikan Kode Penanganan State Keranjang Belanja (CartScreen):

Lokasi Kode: `lib/features/cart/screens/cart_screen.dart` (Baris 94 - 138)

```

// Reaktif terhadap perubahan CartService menggunakan AnimatedBuilder
Widget build(BuildContext context) {
  return AnimatedBuilder(
    animation: _cartService,
    builder: (context, _) {
      // 1. Penanganan Kondisi EMPTY STATE
      if (_cartService.items.isEmpty) {
        return _buildEmptyCart(isDark);
      }

      // 2. Penanganan Kondisi SUCCESS STATE (Render Daftar Item)
      return Column(
        children: [
          Expanded(
            child: ListView.builder(
              itemCount: _cartService.items.length,
              itemBuilder: (ctx, i) => _buildCartItemCard(_cartService.items[i], isDark),
            ),
          ),
          // Kalkulasi Subtotal & Delivery Fee Reaktif
          _buildCheckoutSummaryCard(_cartService),
        ],
      );
    },
  );
}

```

3.2 Studi Kasus 2: Alur Langganan NutriMarket Plus (Subscription / Paywall Flow)

State UI	Kondisi Logika Sistem	Representasi Visual Antarmuka	Lokasi Implementasi Kode
INITIAL	Membaca status langganan <code>isPremium</code> dari profil pengguna di <code>HistoryService</code> .	Jika sudah premium, muncul status aktif mahkota emas & tombol 'Batalkan'. Jika belum, muncul opsi paket.	<code>HistoryService().userProfile.isPremium</code> <code>lib/features/subscription/screens/paywall_screen.dart</code>
PROCESSING	Aktivasi paket langganan: update state lokal, simpan ke <code>LocalStorage</code> , dan upsert ke Supabase.	Proses asinkron dieksekusi dalam handler <code>_activatePremium()</code> .	<code>PaywallScreen._activatePremium()</code> <code>lib/features/subscription/screens/paywall_screen.dart:21</code>
SUCCESS	Status <code>isPremium</code> berhasil diset ke <code>true</code> . Diskon 10% keranjang & AI tanpa batas aktif.	Popup <code>AppToast.success</code> 'Nutri Market Plus Aktif! 👑' dan pembuatan notifikasi sistem.	<code>NotificationService().addNotification()</code> <code>lib/features/subscription/screens/paywall_screen.dart:36-52</code>
CANCELLATION	Pengguna memilih batalkan langganan melalui modal konfirmasi dialog.	Dialog peringatan hilangnya benefit diskon & AI Pro, state <code>isPremium</code> dikembalikan ke <code>false</code> .	<code>PaywallScreen._cancelPremium()</code> <code>lib/features/subscription/screens/paywall_screen.dart:55-105</code>

Cuplikan Kode Alur Langganan (PaywallScreen):

Lokasi Kode: `lib/features/subscription/screens/paywall_screen.dart` (Baris 21 - 53)

```
Future<void> _activatePremium() async {
  // 1. Update State Domain & Local Storage Cache
  final historyService = HistoryService();
  final updated = historyService.userProfile.copyWith(isPremium: true);
  historyService.updateUserProfile(updated, notifyUser: false);
  LocalStorageService().saveUserProfile(updated.toMap());

  // 2. Sinkronisasi Data ke Supabase Cloud Backend
  final supabase = SupabaseService();
  if (supabase.isConfigured && supabase.currentUser != null) {
    try {
      await supabase.client!.from('profiles').upsert({
        'id': supabase.currentUser!.id,
        'is_premium': true
      });
    } catch (e) {
      debugPrint('[Paywall] Error syncing premium: $e');
    }
  }

  // 3. Buat Notifikasi & Feedback SUCCESS ke UI
  NotificationService().addNotification(
    title: 'Nutri Market Plus Aktif 👑',
    message: 'Langganan aktif. Nikmati diskon 10% dan AI tanpa batas.',
    category: NotificationCategory.promo,
    isImportant: true,
  );

  setState(() {});
  if (mounted) {
    AppToast.show(
      context,
      title: 'Nutri Market Plus Aktif! 👑',
      subtitle: 'Selamat! Diskon 10% di keranjang belanja dan Konsultasi AI Pro telah aktif.',
      type: ToastType.success,
    );
  }
}
```

3.3 Studi Kasus 3: Alur Katalog Produk & Autentikasi Login (5 Siklus Lengkap)

Lokasi Kode: lib/features/products/screens/home_screen.dart & lib/features/auth/screens/login_screen.dart

```
// Contoh Multi-State Handling di HomeScreen (lib/features/products/screens/home_screen.dart)
final productRepo = context.watch<ProductRepository>();
final products = productRepo.products;

// 1. Kondisi LOADING: Menampilkan spinner animasi saat fetch API berlangsung
if (productRepo.isLoading && products.isEmpty)
  const Center(child: CircularProgressIndicator(color: AppColors.primary))

// 2. Kondisi EMPTY: Menampilkan visual 'Produk Tidak Ditemukan' jika filter bernilai 0
else if (products.isEmpty)
  EmptyStateWidget(message: 'Belum ada produk untuk kategori ini')

// 3. Kondisi SUCCESS: Menampilkan Grid Katalog Produk
else
  GridView.builder(
    itemCount: products.length,
    itemBuilder: (ctx, i) => FoodCard(product: products[i]),
  );
```


4. RANGKUMAN & CHECKLIST VERIFIKASI KODE SUMBER

Tabel berikut merangkum seluruh poin capaian pembelajaran CPMK 3 beserta referensi file dan nomor baris kode pada proyek **clev_ai**:

Poin	Persyaratan CPMK 3	File Sumber & Letak Kode	Status Implementasi
Poin 1	Pemilihan Pustaka State Management Standar Industri	<code>pubspec.yaml</code> (Baris 12) <code>provider: ^6.1.2</code>	✓ Selesai (Provider standar resmi)
	Registrasi Root State Management	<code>lib/main.dart</code> (Baris 67 - 88) <code>MultiProvider</code> (9 Provider terdaftar)	✓ Selesai (Global provider tree)
Poin 2	Presentation Layer (UI)	<code>lib/features/cart/screens/cart_screen.dart</code> <code>lib/features/subscription/screens/paywall_screen.dart</code>	✓ Selesai (Bebas query DB langsung)
	Domain / Business Logic Layer	<code>lib/features/cart/services/cart_service.dart</code> <code>lib/features/orders/services/order_service.dart</code>	✓ Selesai (Pure business rules & ChangeNotifier)
	Data Layer (Remote & Cache)	<code>lib/data/remote/supabase_service.dart</code> <code>lib/data/local/local_storage_service.dart</code>	✓ Selesai (Supabase & SharedPreferences)
Poin 3	Penanganan Initial & Empty State	<code>lib/features/cart/services/cart_service.dart:46</code> <code>lib/features/cart/screens/cart_screen.dart:98</code>	✓ Selesai (Cache boot & empty cart UI)
	Penanganan Loading State	<code>lib/features/products/screens/home_screen.dart:106</code> <code>lib/features/auth/screens/login_screen.dart:72</code>	✓ Selesai (Circular indicator & async lock)
	Penanganan Success State	<code>lib/features/cart/screens/cart_screen.dart:102</code> <code>lib/features/subscription/screens/paywall_screen.dart:46</code>	✓ Selesai (Reactive render & AppToast success)
	Penanganan Error / Failure State	<code>lib/features/auth/screens/login_screen.dart:108</code> <code>lib/features/orders/screens/checkout_screen.dart:345</code>	✓ Selesai (Try-catch, AppToast error fallback)

Kesimpulan Arsitektur:
Implementasi sistem pada aplikasi **NutriMarket (clev_ai)** telah memenuhi seluruh capaian **Milestone 3 (CPMK 3)**. Pemisahan antarmuka (Presentation), logika bisnis (Domain), dan persistensi data (Data Layer) yang dikombinasikan dengan pustaka **Provider** dan penanganan status deterministik (Initial, Loading, Empty, Success, Error) menghasilkan arsitektur yang modular, reaktif, mudah diuji, dan handal (*production ready*).